性能测试，问题定位与调优

讲师：刘先军

课程安排

- 第一课　性能测试入门
  - 本课程以性能测试为主线，讲解在客户现场，公司内部性能测试中需注意的事项和性能测试的过程方法。
- 第二课　问题定位与解决入门
  - 本课程以问题定位为主，以一次性能测试的过程为主线讲解在性能测试的过程中，如何定位问题，发现问题并解决问题，如何做基础的调优工作。

课堂纪律:

60分钟一课时，休息15分钟

手机震动，避免影响他人

第一课　性能测试入门

目录

- 什么是性能，什么是性能测试
- 性能测试工具介绍
- 目前产品的性能测试做法

软件性能的定义

软件性能是什么?

- IEEE对软件性能的定义是:
  - 软件的固有属性
  - 软件的及时性符合用户要求的程度

不同视角的软件性能体现

用户视角

- 响应时间

系统视角

- 并发用户数量
- 用户操作模式（每秒点击数）

调优视角

- 服务器资源情况
- 应用服务器资源情况
- 数据库资源情况
- 应用时间消耗分布

性能测试的几个基本概念

- 响应时间
- 并发数
- 吞吐量
- 资源利用率
- 场景

响应时间

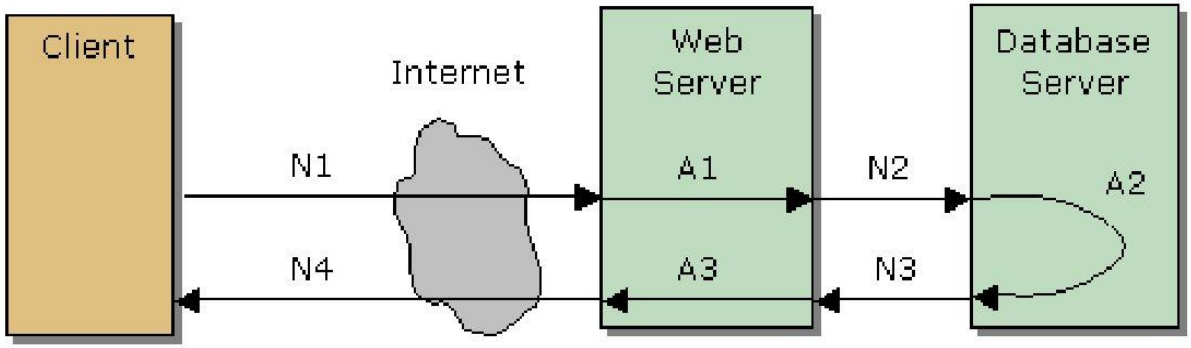
定义：

响应时间指的是从客户端发起一个请求开始，到客户端接收到从服务器端返回的响应结束，这个过程所耗费的时间。

在某些工具中，响应时间通常会称为“**TTLB**”，即“time to last byte”，意思是从发起一个请求开始，到客户端收到最后一个字节的响应所耗费的时间。

响应时间的单位一般为“秒”或者“毫秒”。

响应时间的分解



响应时间=

网络响应时间 + 应用程序响应时间

响应时间 =

(N1+N2+N3+N4)+(A1+A2+A3)

并发数

- 并发数是指同时进行请求的客户的数量
- 业务并发数用于模拟用户的真实负载情况（并发情况是对系统最大的考验）
  - 业务并发数通常具有业务的含义，体现的是用户视角的并发
- 服务端的并发数表明软件在同一时刻收到了多少用户请求
  - 服务端的并发数体现的是应用本身的压力，一般用于查找并发引起的问题时才使用这个概念

确定并发数

- 业务并发数（并发数）一般通过两种方式来获得
  - 通过应用服务器或是应用的日志来获得——可以用“在线用户数”体现
  - 通过估算获得——在应用尚未被使用时，可以通过估算获得这类数据
- 服务端并发数同样可以通过分析日志和估算来获得
  - 通过数据库服务器、应用服务器等的日志来获得“同时进行某个操作的并发数量”
  - 通过估算，或是以往的经验来判断

并发数估算的简单原则

对于OA类应用，可以简单的用10％－20％的“用户数”作为估算的并发用户数

但要注意的是，在估算用户数时，必须考虑不同的场景对并发用户数的影响

吞吐量

定义：吞吐量指的是单位时间内处理的客户端请求数量。

通常情况下，吞吐量用请求数/秒或者页面数/秒来衡量。

从业务角度看，吞吐量也可以用访问人数/天

或者页面访问量/天 来衡量。

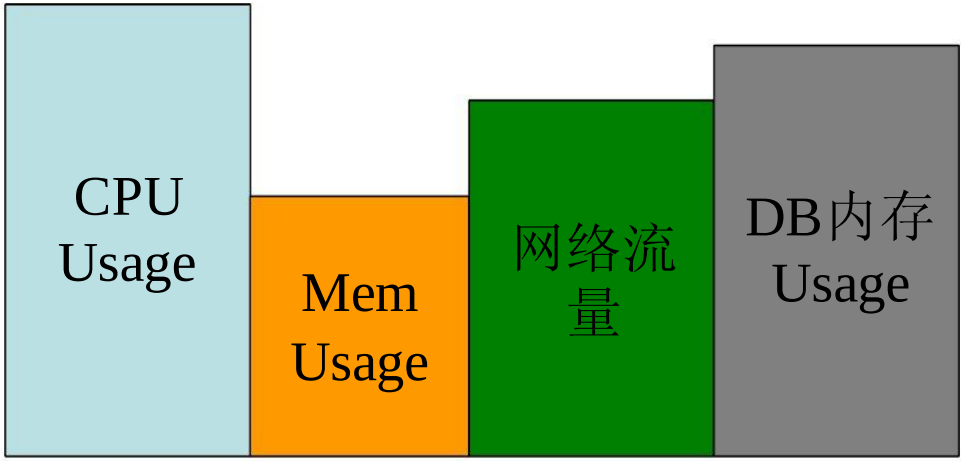
资源利用率

定义：资源利用率指的是对不同系统资源的使用程度，例如服务器的CPU（s），内存，网络带宽等。

资源利用率通常以占用最大值的百分比 n %来衡量。

PRIMETON · 普元

横向对比的资源利用率



场景

场景是应用运行时的一个剖面

一般来说，一个场景可以被表述如下：

x％的用户在操作A业务，y％的用户在操作B业务，z％的用户在操作C业务

场景不同就意味着系统在被以不同的方式使用，在不同的场景下，很可能系统的性能表现就会不同

性能测试的分类

性能测试（Performance Testing）
负载测试 （Load Testing）
压力测试 （Stress Testing）
配置测试 （Configuration Testing）
并发测试 （Concurrency Testing）
容量测试 （Volume Testing）
可靠性测试 （Reliability testing ）
失败测试 （Failover Testing）

Performance Test

定义：

通过模拟生产运行的业务压力量和使用场景组合测试系统的性能是否满足生产性能要求。如以实际投产结构测试，求出最大的吞吐量与最佳回应时间以保证上线的平稳，安全等

Load Testing

定义：

通过在被测系统上不断增加压力，直到性能指标例如响应时间超过预定指标或者某种资源已经达到饱和状态。这种测试可以找到系统的处理极限，为系统调优提供数据。

也被称为Scalability Testing

Stress Testing

定义：

测试系统在一定饱和状态下，例如CPU、内存在饱和使用饱和情况下，系统能够处理的会话能力，以及系统是否会出现错误。

Configuration Testing

定义

通过测试找到系统各项资源的最优分配原则。

Concurrency Testing

定义：

测试多个用户同时访问同一个应用、同一个模块或者数据记录时是否存在死锁或者其他性能问题。

Volume Testing

定义：

测试系统能够处理的最大会话能力。

Reliability Testing

定义：

通过给系统加载一定的业务压力（例如资源在70-90%的使用率）的情况下，运行一段时间。

Failover Testing

定义

对于有冗余备份和负载均衡的系统，通过这样的测试来检验如果系统局部发生故障用户是否能够继续使用系统，用户将受到多大的影响。

性能测试的应用领域

验证能力

能力规划

性能调优

缺陷修复

验证能力

- 在指定环境和负载下能满足预期的设计要求
- 常用于系统上线后对系统性能能力的验证
- 特点
  - 测试环境和负载有明确定义
  - 非探索性的
  - 采用Performance Testing和Reliability Testing

能力规划

- 考虑以下的问题
  - 在要求平均响应时间小于5秒的前提下，你如何判断系统是否能够支持200万用户万用户/天的访问量？
  - 如果你的公司计划在将来的两个月内把业务量提高**200%**，你如何判断公司的Web应用系统是否需要升级服务器的配置或者是增加服务器的数量？
  - 根据历史信息分析，在今年高考查分期间公司的网站系统将会承受去年同期5倍的访问量，以目前的现状，能承受吗？

探索性测试

用于能力规划的测试方法

- Configuration Testing
- Load Testing
- Reliability Testing
- Failover Testing

对第一个问题：

Load Testing

对第二个问题：

Configuration Testing（CPU、内存、网络环境）

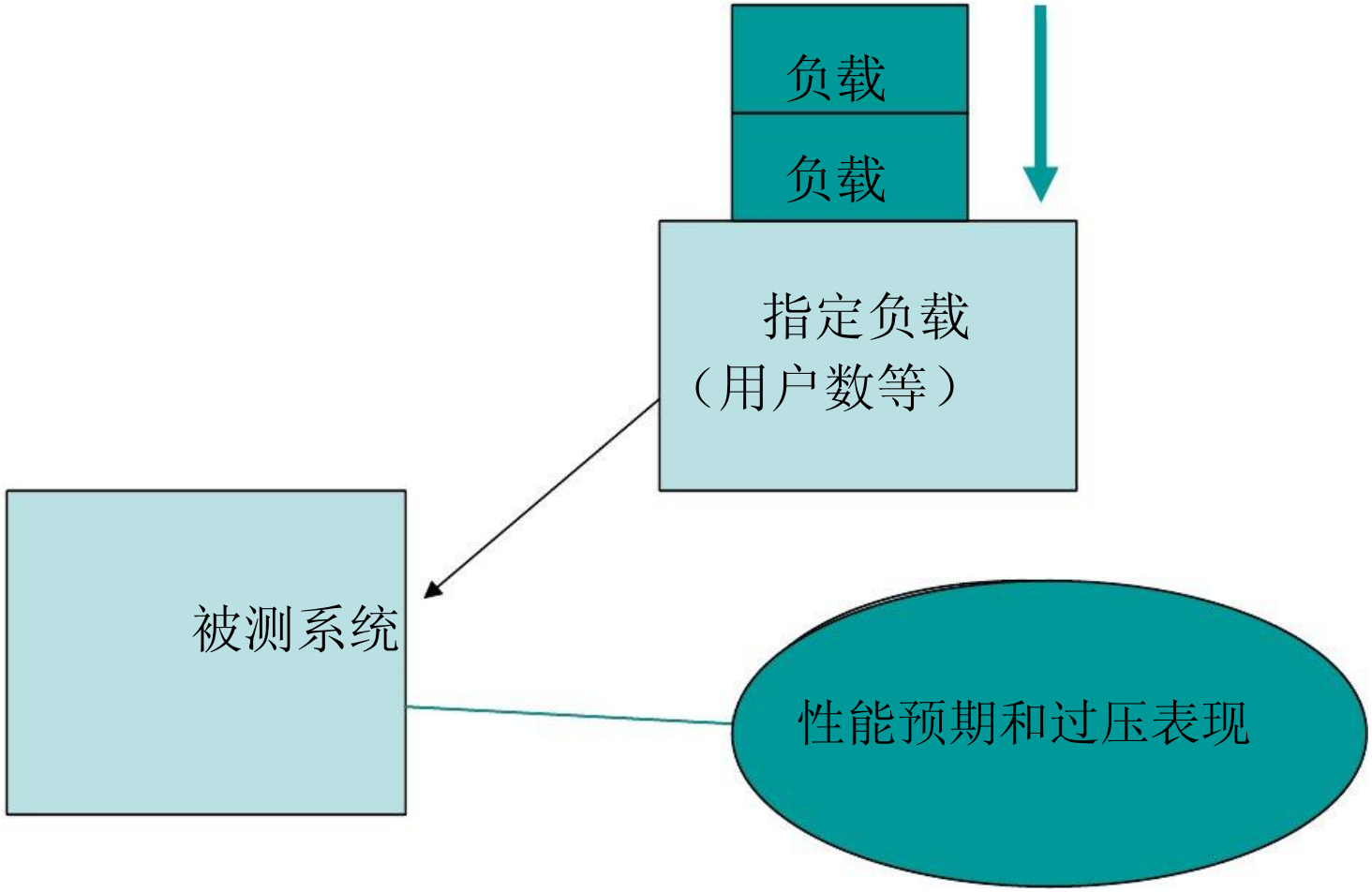
对第三个问题：

Load Testing

Reliability Testing

Failover Testing

探索性测试



性能调优

增加服务器、提高服务器和网络的配置是提高性能的办法，但性能测试能提供更合理的性能调优依据

对系统和应用程序进行仔细的调优，以提高响应时间、吞吐量和资源利用率的性能指标。

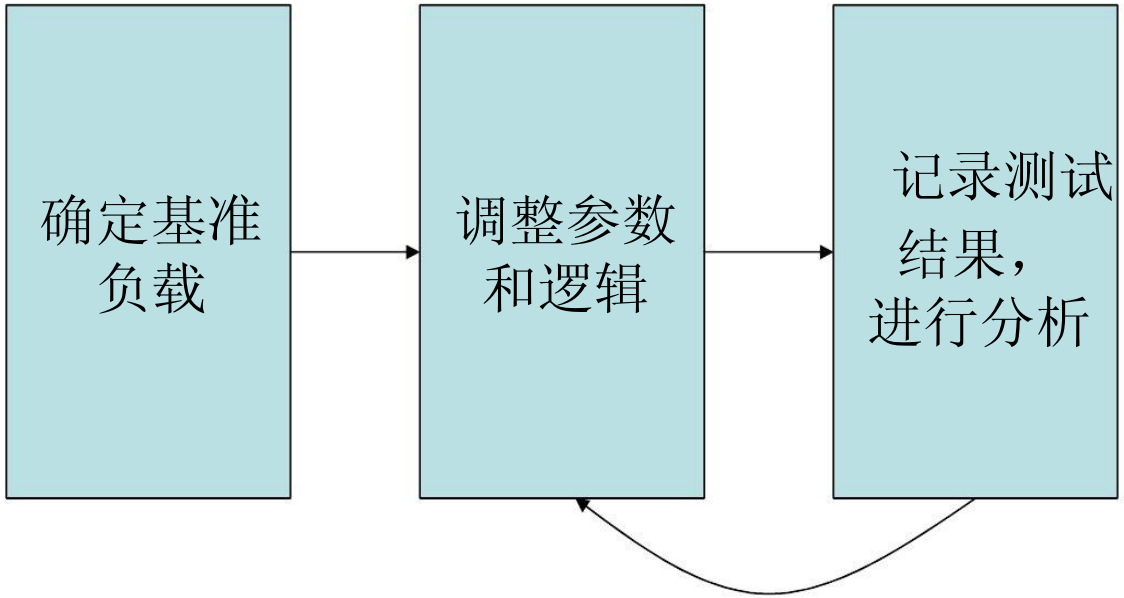
调优对象

应用程序

系统平台设置

Web服务器设置

调优的方法



调优的要点

- 保持同样的负载
- 一次只调整一个或少数几个参数或业务逻辑
- 需要对每次测试结果进行分析，确定调整是否带来了效果
- 达到预期目标后测试结束

缺陷修复

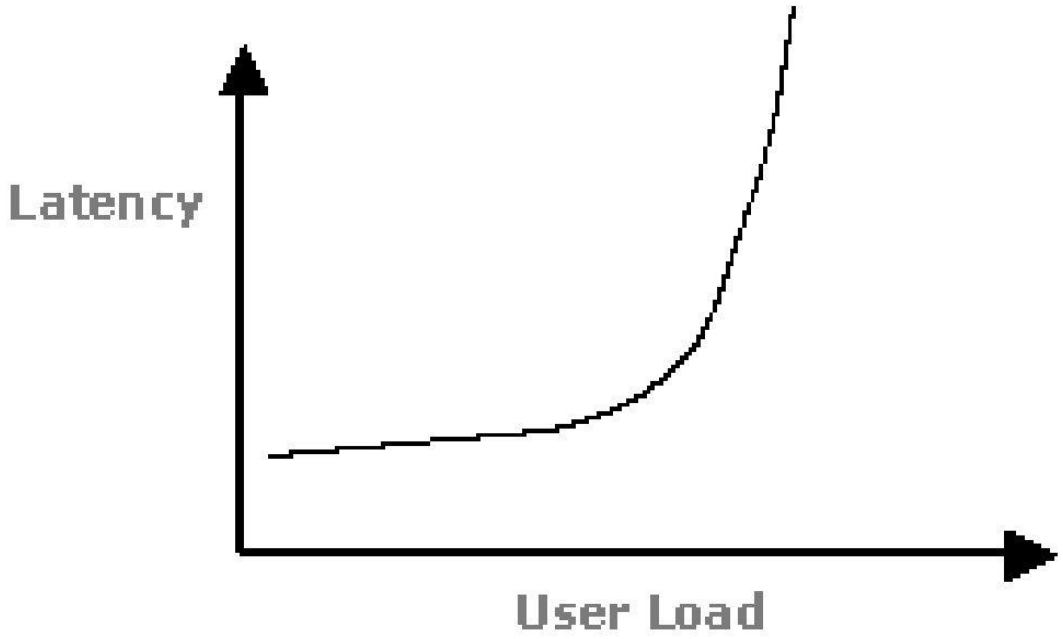
利用性能测试的方法发现在特定情况下的缺陷

- 并发中的死锁 （Concurrency Testing）
- 持续运行过程中的内存泄漏 （Reliability Testing）
- 极端情况下的不正常行为 （Failover Testing）

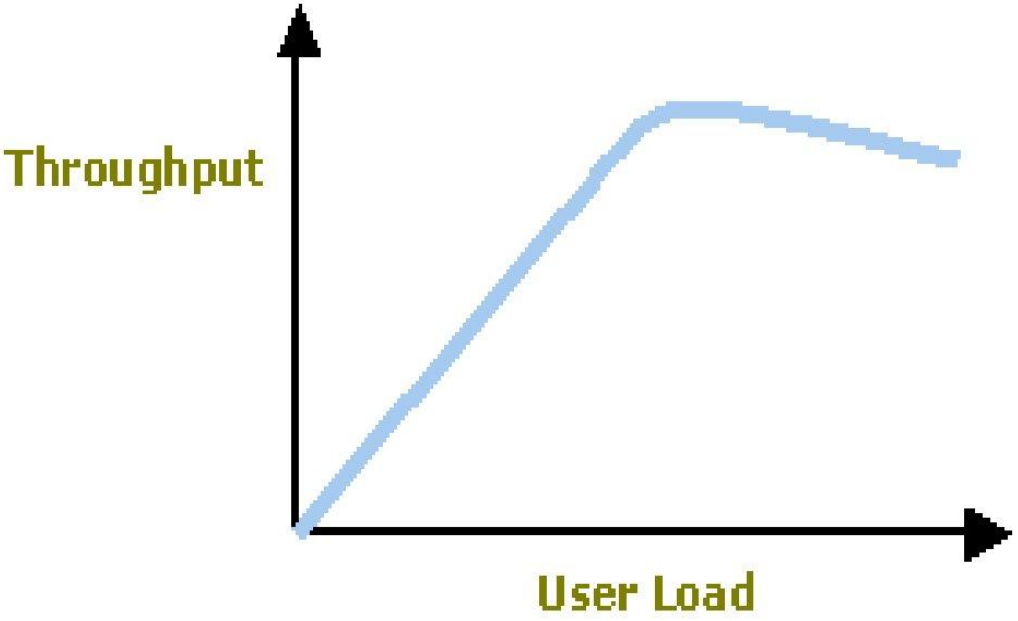
性能测试结果分析

- 性能测试数据需要综合分析，单从某个数据无法得出准确的结论
- 测试数据分析时，采用图表方式比较直观

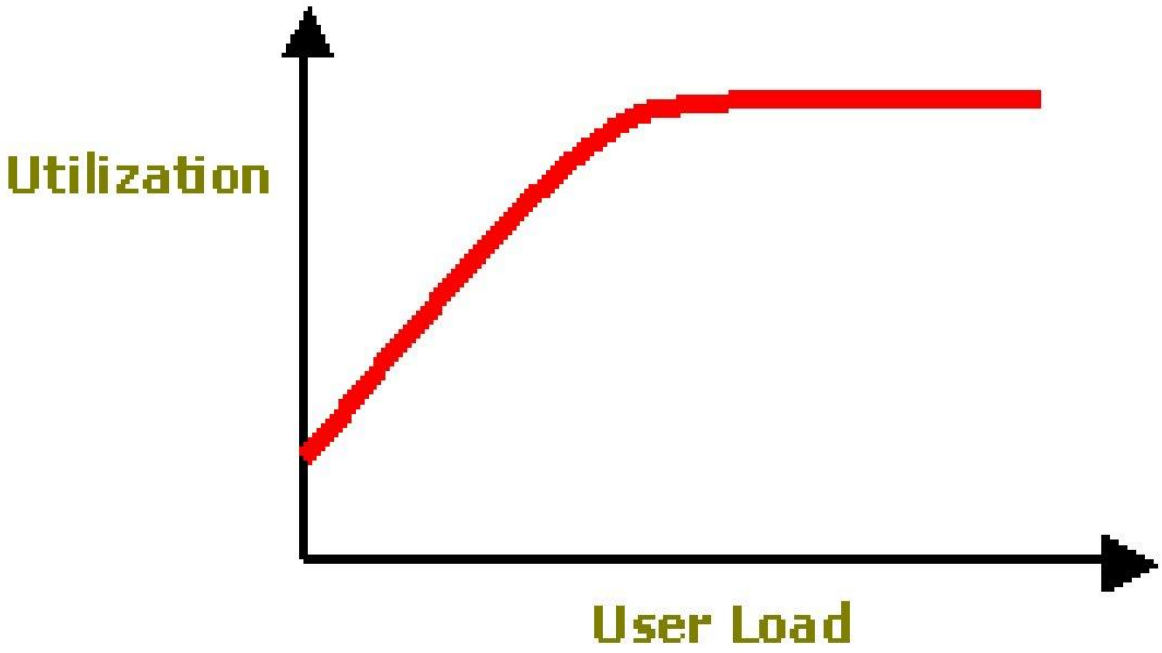
性能测试结果图形的“定式”



负载—响应时间曲线图



负载－吞吐量曲线图



负载—资源利用率曲线图

软件性能测试的特点

性能测试的目标较难确定

性能测试具有较大的环境依赖性

性能测试是一个系统性的工作

性能测试一般需要工具的支持

B/S和C/S系统性能测试

- 影响性能的因素不同
  - B/S系统：服务器、数据库、网络、应用服务器、应用
  - C/S系统：服务器、数据库、网络、应用
- C/S结构的应用通信协议比较多样
  - B/S系统：HTTP、HTTPS协议
  - C/S系统：数据库协议、基于TCP的协议、自定义协议

软件性能测试的一般过程

- 测试计划
  - 测试目标确定
  - 测试原则和策略
  - 测试工具选择
- 测试设计
  - 测试环境设计
  - 测试用例设计
  - 测试数据设计
  - 测试工具脚本设计

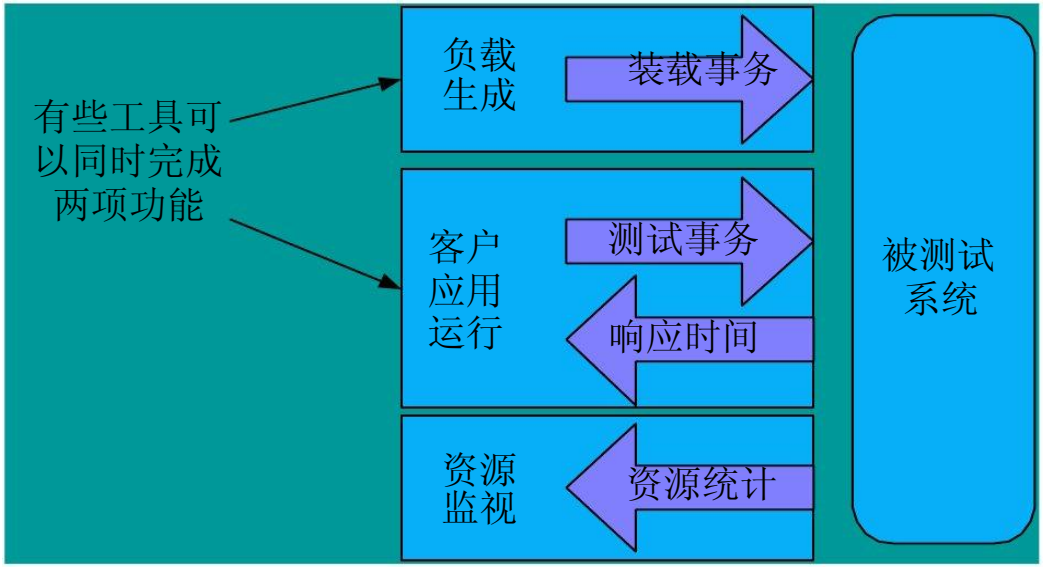
软件性能测试的一般过程

- 测试实施
  - 测试环境维护
  - 测试执行
  - 测试结果记录
- 测试结果分析
  - 数据处理和分析
  - 测试报告

目录

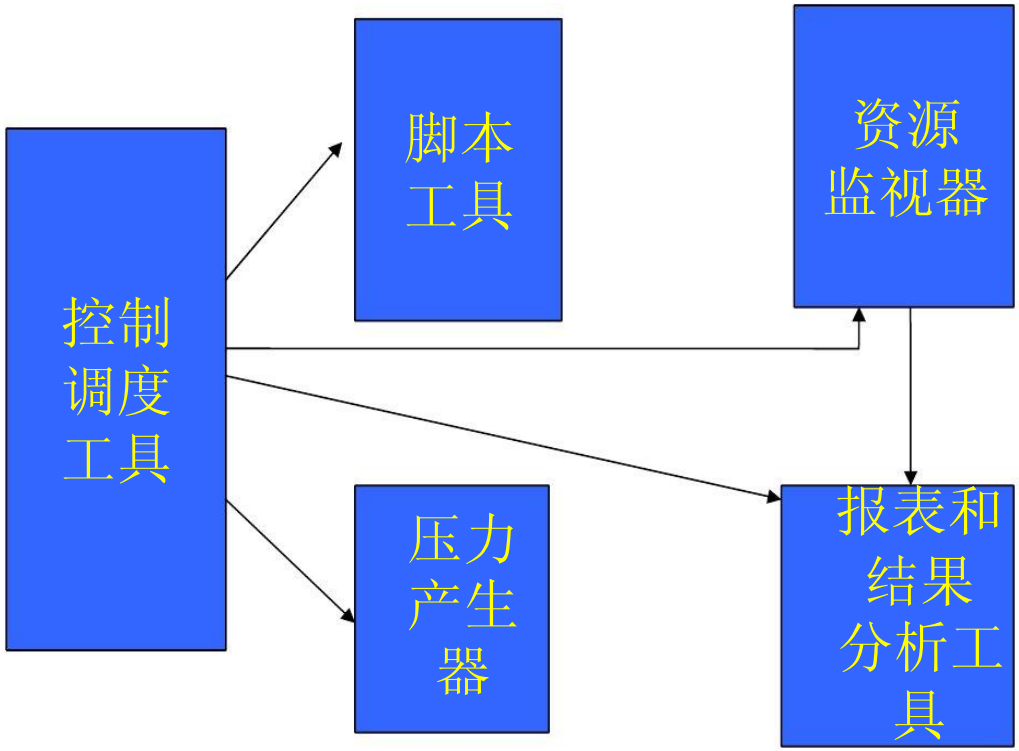
- 什么是性能，什么是性能测试
- 性能测试工具介绍
- 目前产品的性能测试做法

性能测试工具原理



- 基于功能测试工具，需要增加加载和监测分析手段；

压力测试工具原理



MI LoadRunner介绍

- LoadRunner是专用于性能测试的工具，根据2003年对欧洲市场的统计，LR占了约一半的市场份额
- LoadRunner功能强大，在易用性方面相比同样功能的软件来说，处理得比较好
- LoadRunner能够与MI的其他产品，如QualityCenter、WinRunner等很好集成

LoadRunner的功能特点

清晰的分层管理方式

强大的脚本语言

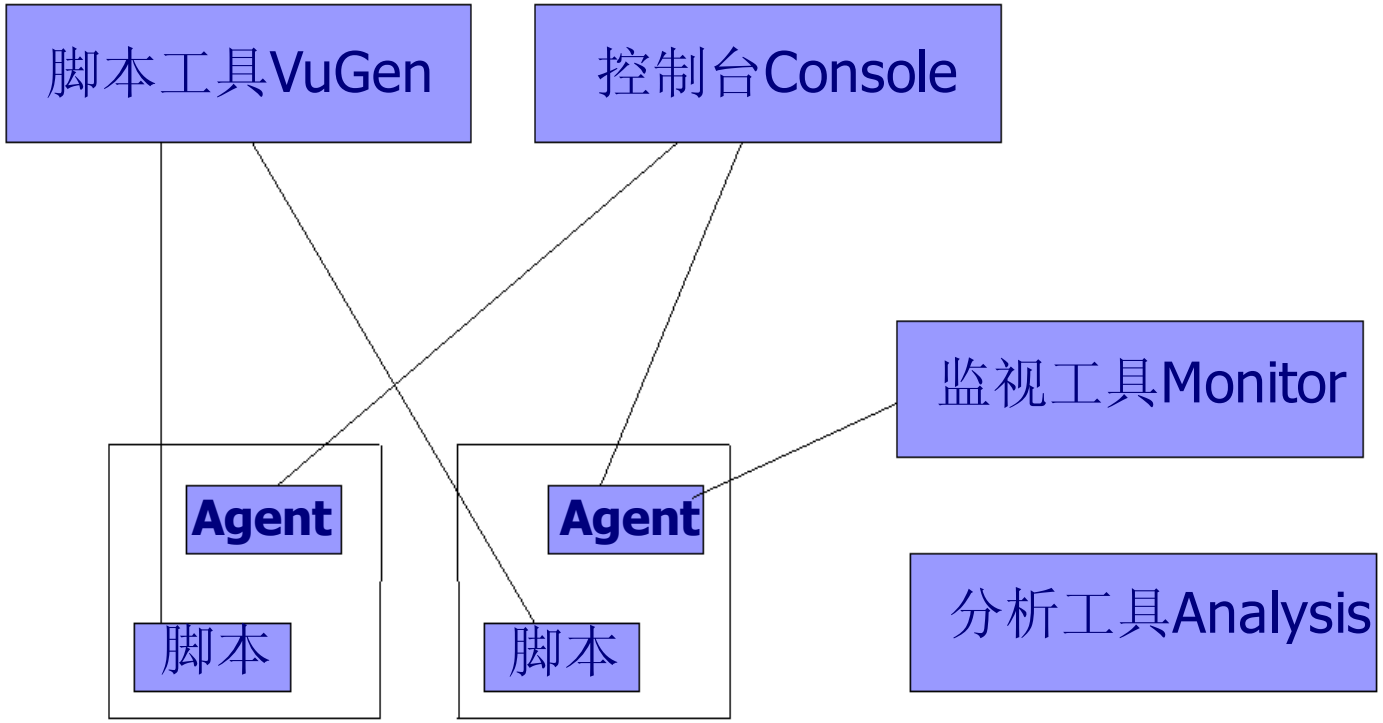
丰富的协议支持

强大的数据分析和报告系统

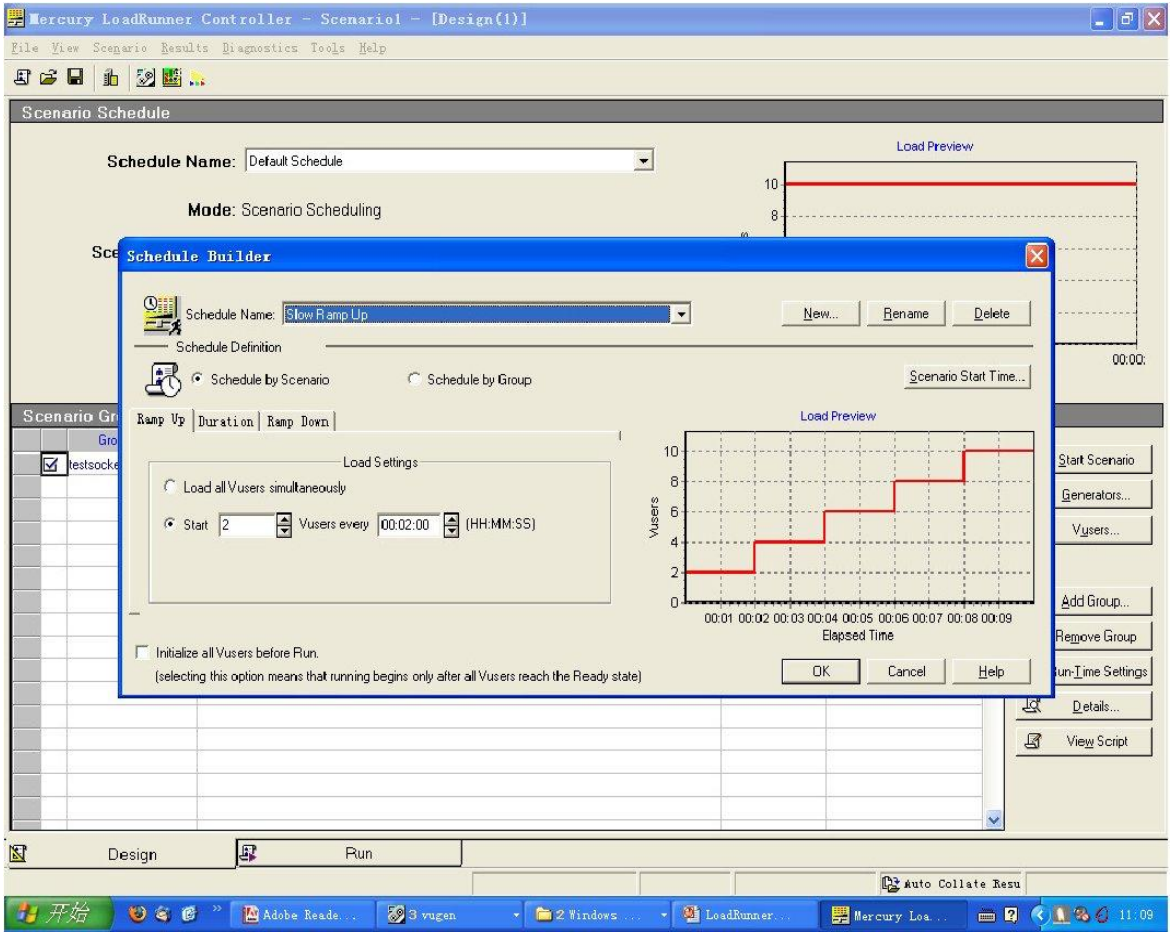
支持IP伪装

增加了专门的Tuning Console (LoadRunner 8.0)

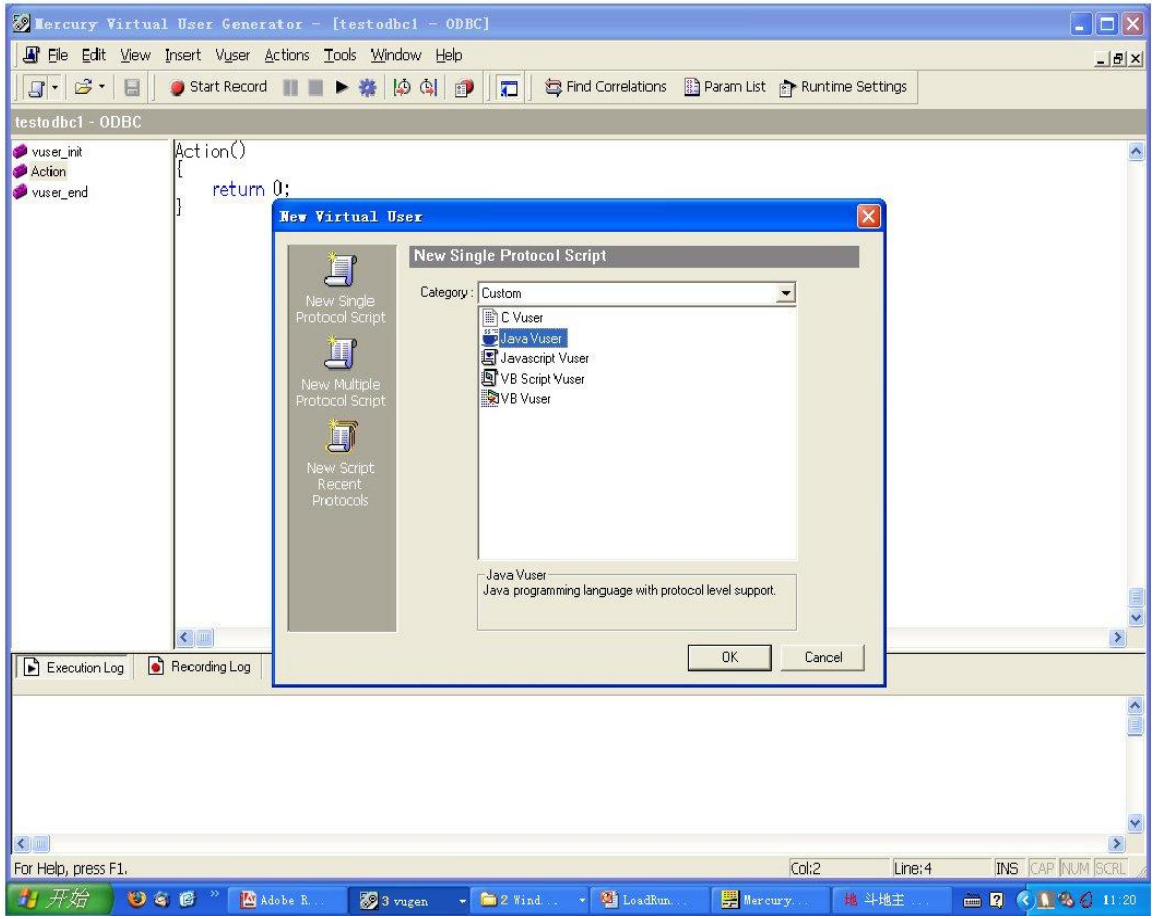
LoadRunner的基本结构



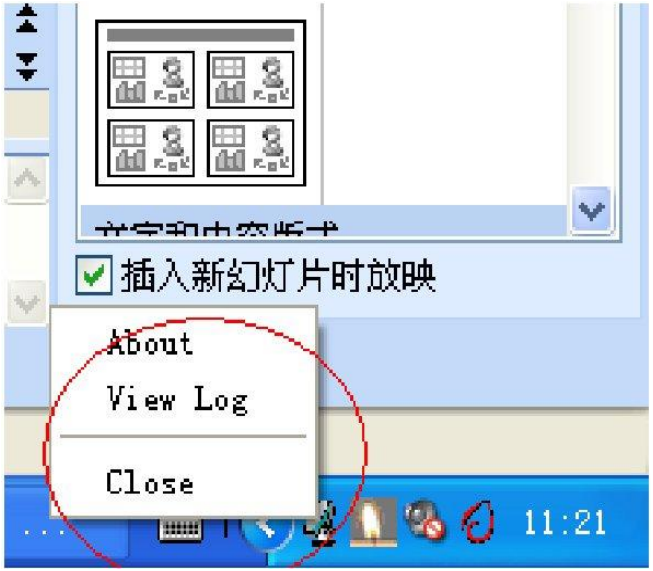
控制台（Console）


Mercury LoadRunner Controller - Scenario1 - [Design(1)]
File View Scenario Results Diagnostics Tools Help
Scenario Schedule
Schedule Name: Default Schedule
Mode: Scenario Scheduling
Load Preview
Schedule Builder
Schedule Name: Slow Ramp Up
New... Rename Delete
Schedule Definition
Schedule by Scenario
Schedule by Group
Scenario Start Time...
Ramp Up Duration Ramp Down
Load Settings
Load all Vusers simultaneously
Start 2 Vusers every 00:02:00 (HH:MM:SS)
Vusers
Elapsed Time
Initialize all Vusers before Run.
(selecting this option means that running begins only after all Vusers reach the Ready state)
OK Cancel Help
Start Scenario
Generators...
Vusers...
Add Group...
Remove Group
Run-Time Settings
Details...
View Script
Design
Run
Auto Collate Resu

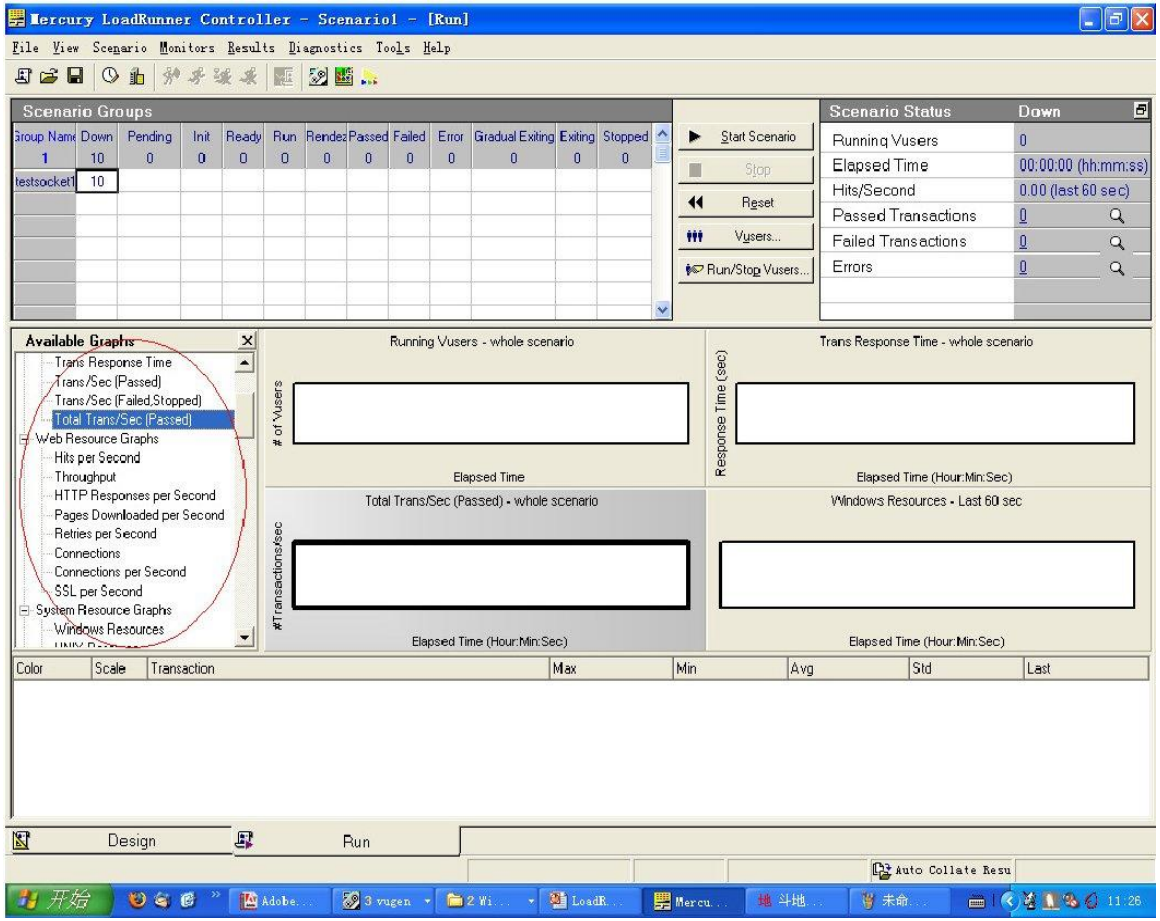
脚本工具（VuGen）


Mercury Virtual User Generator - [testodbc1 - ODBC]
File Edit View Insert Vuser Actions Tools Window Help
Start Record
Find Correlations
Param List
Runtime Settings
testodbc1 - ODBC
vuser_init
Action
vuser_end
Action()
{
return 0;
}
New Virtual User
New Single Protocol Script
New Multiple Protocol Script
New Script Recent Protocols
Category : Custom
C Vuser
Java Vuser
Javascript Vuser
VB Script Vuser
VB Vuser
Java Vuser
Java programming language with protocol level support.
OK
Cancel
Execution Log
Recording Log
For Help, press F1.
Col:2
Line:4
INS
CAP
NUM
SCRL

代理（Agent）

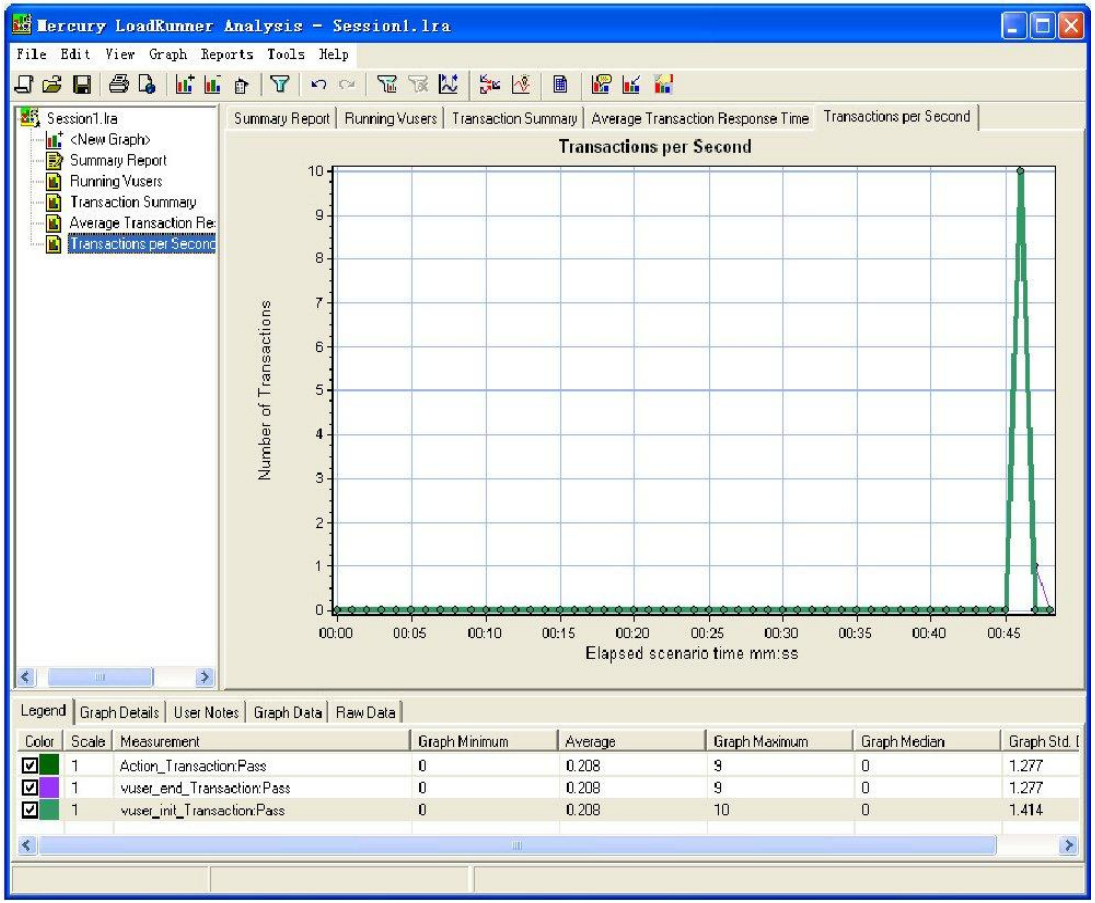


监视器（Monitor）



PRIMETON · 普元

报表分析器（Analysis）



其他工具

IP地址伪装

配置工具

LoadRunner常用术语

Transaction（事务）

Vuser_init、Action、vuser_end、 Iteration（迭代）

VUser（虚拟用户）

Rendezvous（集合点）

Scenario（场景）

Load Generator（负载发生器）

Think Time

Transaction

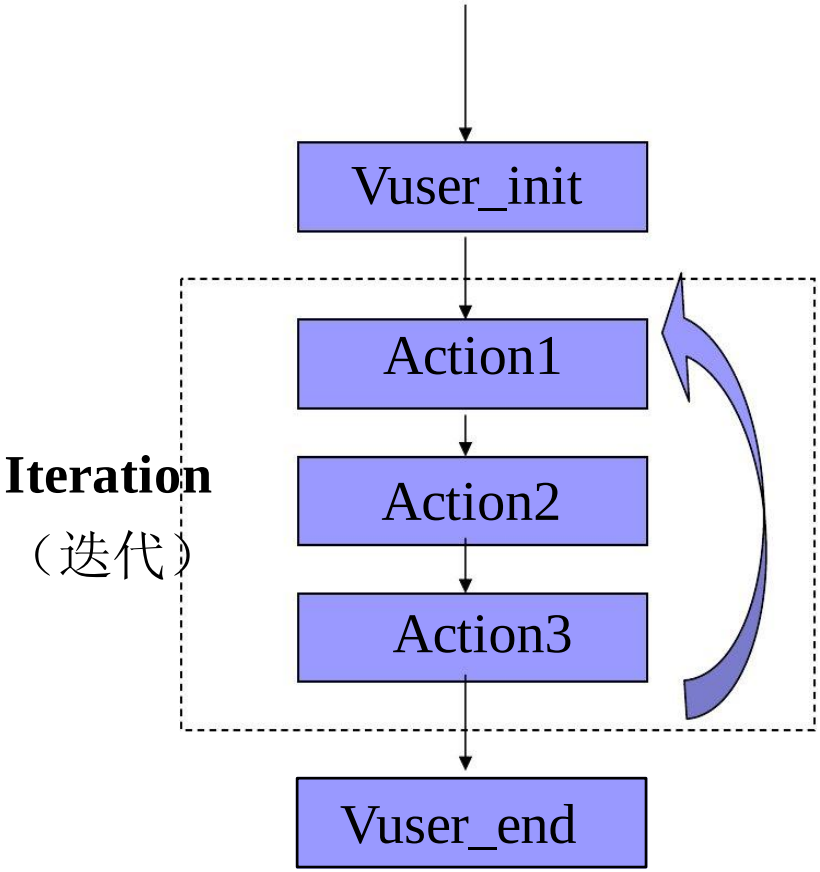
- Transaction由一组可被单独计算时间的请求组成
  - Transaction标识需要计算其响应时间的请求组
  - Transaction的响应时间计数在最终生成的报告中体现
  - 注意：如果在Transaction中包含了think time，则根据你的设置，think time也会包含在最终得到的结果中

脚本的分段

Vuser_init段

Action段

Vuser_end段



Action中的Block

- Block是一个“块”的概念，该概念可用来组织不同的Action，达到更好地模拟实际用户操作的效果
  - Block可以包含Action
  - Block可以嵌套，并可以在一个迭代过程中设定Block的调用几率

VUser（虚拟用户）

- LoadRunner运行场景时，Controller可以调度产生许多VUser（虚拟用户）
  - 从业务的角度理解，一个VUser就是一个参与性能测试的执行者
  - VUser可被调度为进程或是线程
  - VUser可以通过参数、集合点等方式进行协作

Rendezvous（集合点）

- 集合点是在脚本中标识的一个用于对系统施加并发压力的点
  - 集合点必须在场景中设置策略后才有效
  - 集合点只有在考察系统是否存在并发问题时才会使用到，在一般的性能测试中不需要使用到

Scenario

在LoadRunner中，Scenario指根据性能测试模型建立的用于测试的方案，一个Scenario包括

- 虚拟用户的数量及其在各脚本间的分配
- 测试执行时间以及虚拟用户增长和退出的模式
- 思考时间策略、集合点的策略
- 监控器设置

Load Generator（负载产生器）

- Load Generator是指用于产生性能测试时所需负载的机器，该机器上必须安装和运行Load Machine所需的程序（如Agent等）
  - Load Machine通过Controller来调度和使用
  - Load Machine可以是运行Windows的机器，也可以是运行Unix的机器（安装和运行Unix版本的Load Machine部件）

Think time（思考时间）

- Think time是体现在脚本请求之间的一个等待时间
  - 有些书上建议忽略思考时间，并认为这是一个比较好的方法，其实不然
  - 思考时间具有业务上的含义，通过思考时间才能对Hits per second进行控制

LoadRunner支持的脚本语言

C语言脚本

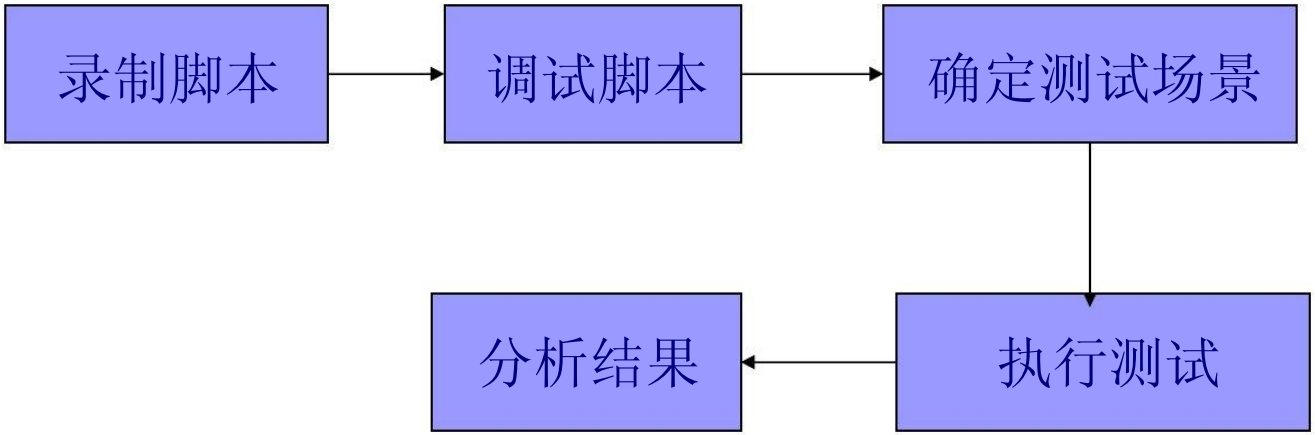
Java语言脚本

JavaScript语言脚本

VB语言脚本

VBScript语言脚本

LoadRunner的应用步骤



以WEB应用的录制为例

- 选择WEB协议
- 设置录制选项（Recording Options）
  - 采用HTML方式或是URL方式录制
  - 设置浏览器（3种不同方式指定）
  - 设置录制代理
  - 其他设置
  - 设置是否使用录制时自动关联

HTML_based VS URL_based

- HTML_based方式录制得到的是以HTML方式描述的脚本，脚本的一条语句通常就对应了一个具体的操作，在大部分情况下应该使用这种方式
- URL_based方式录制得到的是所有的URL请求，只在下列情况下才使用这种方式
  - 应用是通过HTTP协议实现的非WEB应用
  - 应用使用的Applet与服务端之间有交互

设置浏览器

设置使用缺省浏览器

设置使用指定的浏览器（路径指定）

设置通过手工方式启动浏览器

设置录制代理

设置直接连接

设置通过IE中设定的代理进行录制

设置手工指定录制代理

其他设置

是否录制思考时间

是否假设在每次迭代后清除上下文

是否为每个页面产生针对标题的 web_reg_find 函数

是否需要支持UTF-8编码

是否使用录制时自动关联

- 是否允许录制时自动关联
- 设置录制时自动关联的规则
  - 根据特征字符串进行自动关联

开始录制

- 选择应用
  - 指定URL入口
- 执行操作
  - 按照用例进行操作
- 停止录制

Script方式和TreeView方式

- Script方式是最方便的对脚本进行编辑的方式
  - 脚本以代码的形式展现
  - 适合进行参数化和修改代码操作

脚本初步释义

脚本中的每条语句均对应对服务端的操作

- GET
- POST

从业务的角度理解，基本上，一条语句对应一个用户的操作（HTML_based）

脚本的回放

- 点击工具条上的“Run”按钮或是从菜单中选择“VUser”－“Run”可以执行回放
- 回放意味着
  - 虚拟一个客户端
  - 虚拟的客户端按照脚本描述向服务端发出请求

回放选项设置

Run Logic设置

Think time设置

浏览器模拟设置

代理设置

其他设置

Run Logic设置

设置迭代次数

设置Block及其取值的概率

调整Action的执行顺序

Think Time设置

- 设置是否在回放时使用Think time
- 设置Think time在回放时的行为
  - 按照录制的时长等待
  - 等待固定的时间
  - 等待一定范围内的随机时间
- 最佳实践
  - 设定平均的think time时长
  - 回放时使用50％－150％的随机时间

浏览器模拟设置

是否模拟浏览器cache

是否每次迭代模拟一个新用户

是否在每次迭代时清空cache

调整脚本区域（Section）

脚本的vuser_init和vuser_end区都只会被执行一次，因此一般在vuser_init中放置初始化和登录部分，在vuser_end中放置只执行一次的结束处理语句

需要重复执行的部分放置在Action部分

对部分类型的脚本，可以设置多个Action

参数化

目的：同一个脚本的Action部分需要运行多次，但要求每次迭代时输入不同数据，此时只能用参数化方式解决

例子

- 被测系统限制不能输入重复数据
- 为方便识别，每次用不同数据区分

参数化的具体做法

从菜单上选择“Insert”——“New Parameter”

在脚本编辑窗口中选中需要参数化的部分，点击鼠标右键，选择“Replace with a Parameter”

Parameter的不同类型

Date/Time：每次用当前时间替换，格式可设置

File、Table：从指定的文件/表中每次获取数据

Group Name、Iteration Number、Load Generator Name、Vuser ID

Random Number、Unique Number

User Defined Function

Parameter的取值设置

- 取值方式
  - 顺序（Sequence）
  - 随机（Random）
  - 唯一（Unique）
- 改变取值的时机
  - 每次迭代（Each Iteration）
  - 每次出现（Each Occurrence）
  - 只改变一次（Once）

为什么需要Correlation

- Correlation实际上是从Req或Reponse中找到特征字符串，在Response或Req中使用
- 许多“聪明的”应用服务器都使用特征字符串匹配Req和Response对，例如SessionID
- 纯粹录制生成的脚本只是忠实记录了Send和Receive的数据，如果有运行过程中的数据变化，在回放过程中无法正常工作

自动处理Correlaction

自动Correlaction处理只能针对使用HTTP协议、部分Client/Server协议的脚本，针对Socket协议的脚本不能使用自动Correlation方式

自动Correlation的原理

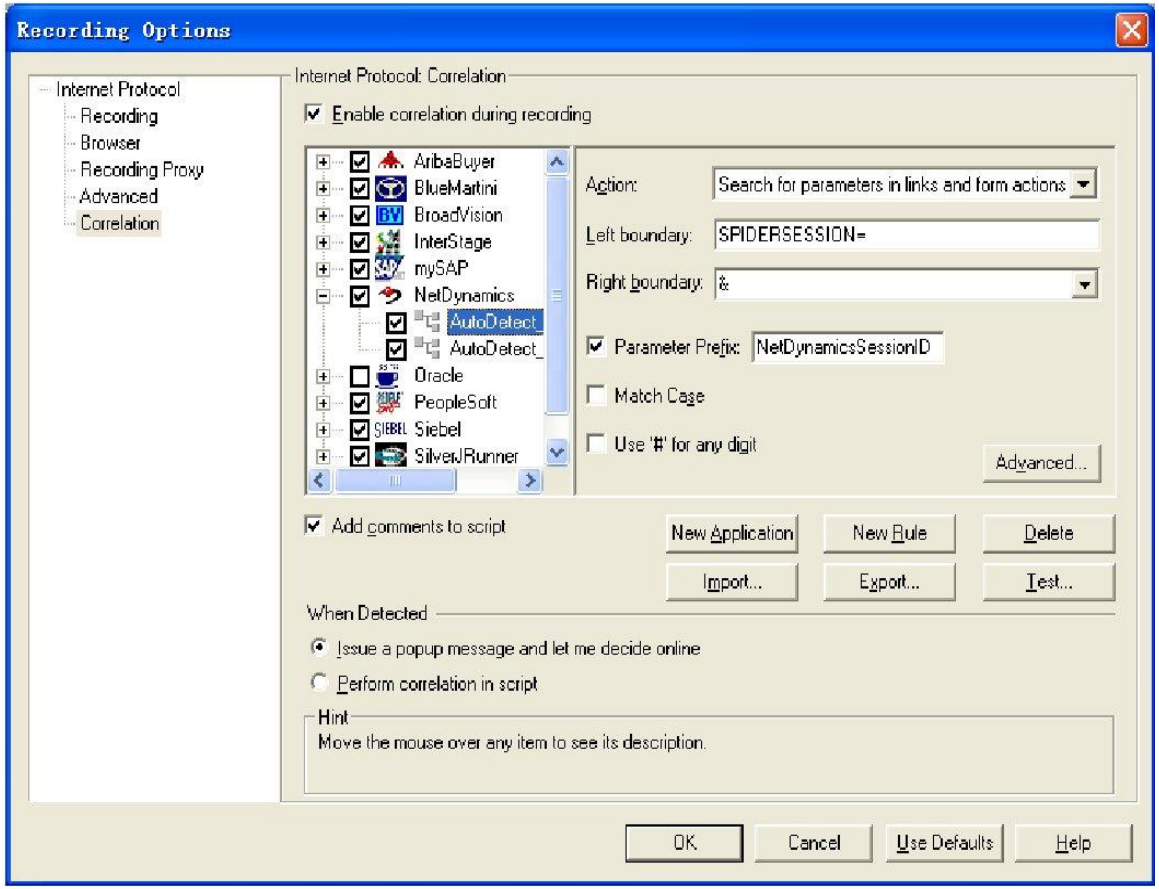
LR内置了部分常用应用服务器和框架产生的特征字串，通过在脚本中搜索特征字串进行替换处理Correlation

自动Correlation的操作步骤

Enable Correlation During Recording

录制完成后，通过“Vuser”——“Scan Correlation”

自定义Correlation规则



手工Correlation

使用LR的函数进行Correlation

- **web_reg_save_param**
- Lrs_save_param

测试场景

手工测试场景（Manual Scenario）

面向目标的测试场景（Goal-Oriented Scenario）

手工测试场景

向场景中添加脚本

设置Load Generator主机，分配负载

设置RunTime Setting

设置监视器

开始运行

分析结果

面向目标的测试场景

- 向场景中添加脚本
- 设置Load Generator主机，分配负载
- 设置目标
- 设置监视器
- 设置RunTime Setting
- 开始运行
- 分析结果

场景与脚本的Runtime Setting

- 场景中针对脚本的Runtime Setting会覆盖脚本原先的Runtime Setting
- 但在场景中针对脚本的Runtime Setting信息仅保存在场景中，不会对脚本本身造成影响

设置虚拟用户的增长和消退模式

- 虚拟用户增长模式
  - 一般建议不要将用户同时增加上去
  - 建议，每30秒增加2－4个虚拟用户是比较合适的速度
- 虚拟用户消退模式
  - 对手工场景而言，可直接设置为“运行完成即停止”

场景运行时长设置

可直接指定场景运行的时长

可根据Runtime Setting中指定的迭代次数控制场景运行时长

建议：一般情况下，使用第一种方式

设置监视器

切换到Scenario的“Run”页面

选中左侧的“Available Graph”

在右侧图形上单击右键

选择“Add Measurements……”

Windows的主要性能计数器

Memory

Process

Processor

Physical Disk

Network Interface

System

Unix的主要性能计数器

Memory

Process

Processor

Physical Disk

Network Interface

System

内存分析方法

- 关注最大可用内存
- 注意Pages/sec、Pages Read/sec和Page Faults/sec的值
- 根据Physical Disk中的某些值协助分析
  - Page Reads/Second
  - %Disk Time
  - Average Disk Queue Length

内存分析方法

可用内存不足——内存瓶颈

如果Pages/sec持续大于几百，很可能有内存问题，Windows平台上Pages Read/sec的阈值一般为5

如果Pages Read/sec很低，同时 %Disk Time 和 Average Disk Queue Length的值很高，则可能有磁盘瓶径。但是，如果队列长度增加的同时 Pages Read/sec并未降低，则是由于内存不足

处理器分析方法

首先关注总的CPU的使用率

其次查看每个CPU的使用率

考虑Process Queue Length的影响

考虑网络问题

磁盘I/O分析方法

计算每磁盘的I/O数

与Processor\Privileged Time合并进行分析

根据Disk sec/Transfer进行分析

计算每磁盘的I/O数

RAID 类 型	计 算 方 法
RAID 0	(Reads + Writes) / Number of Disks
RAID 1	(Reads + 2 * Writes) / 2
RAID 5	[Reads + (4 * Writes)] / Number of Disks
RAID 10	[Reads + (2 * Writes)] / Number of Disks

磁盘I/O分析

如果在Physical Disk计数器中，如果只有%Disk Time比较大，其他值都比较适中，硬盘可能会是瓶颈。若几个值都比较大，且数值持续超过80%，则可能是内存泄漏

一般来说，我们定义Disk sec/Transfer数值小于15 ms为Excellent，介于15 ms到30 ms之间为良好，30 ms至60 ms之间为可以接受，超过60 ms则需要考虑更换硬盘或是硬盘的RAID方式了

J2EE应用服务器常用计数器

服务器CPU、内存状况
服务器IO状况
服务器文件句柄和连接句柄使用情况
JVM内存使用状况
请求队列（Request Queue）长度
超时请求队列长度
转发服务器队列长度（集群方式）
点击数

数据库服务器常用计数器

服务器CPU、内存使用状况
数据库CPU、内存使用状况（PGA等）
表扫描
索引使用状况
锁和死锁数量
缓存命中情况
磁盘IO情况

Windows平台上的性能监控工具

Perfmon工具

- 可用于监控windows平台上的所有性能指标
- 可自行编写脚本调度运行

Unix/Linux 平台上的性能监控工具

vmstat

iostat

sar

Top

rstatd

运行时设置

迭代次数

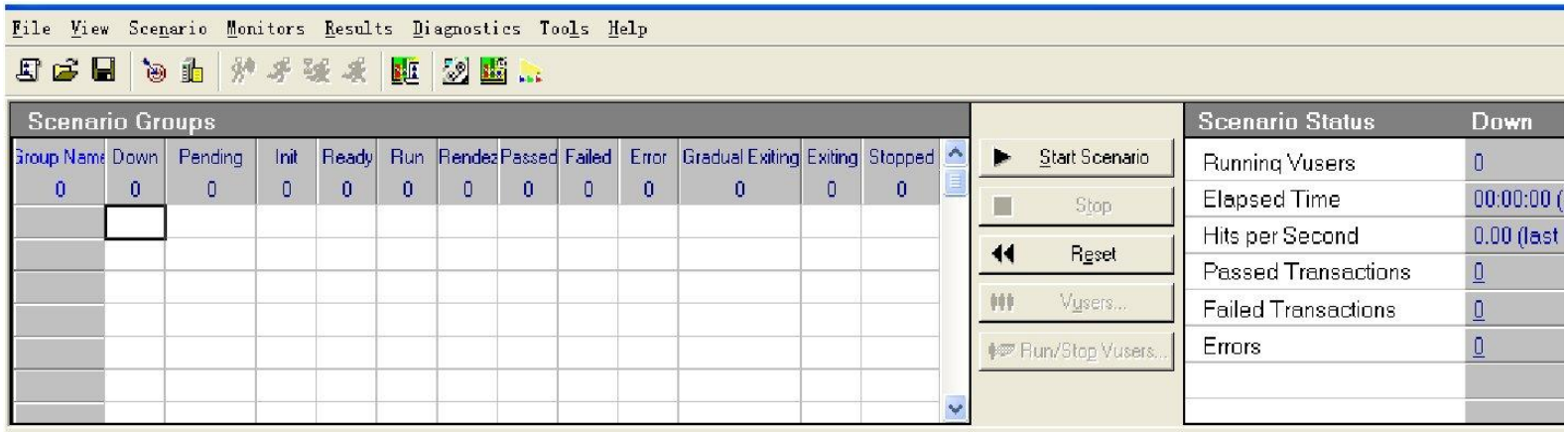
是否写执行过程日志

是否忽略Think Time

开始执行

通过菜单或是工具条按钮开始执行场景

通过“Run”页面监控执行过程



Apache ab 工具

基本命令方式

ab -c 请求数 -n 线程数 网址/目标网页

例如：**ab –c *100* –n *10* *http://www.google.cn/***

其他常用命令参数

-p postfile //包含了需要POST的数据的文件.

-T content-type //POST数据所使用的Content-type头信息。

-C attribute //-C cookie-name=value 对请求附加一个Cookie行。 其典型形式是name=value的一个参数对。此参数可以重复。

-H attribute //向请求中加入指定的头属性信息，例如：‘Accept-Encoding: gzip’

-A attribute //加入基本WWW认证

-X proxy:port //使用的代理服务器和端口

-k //使用HTTP KeepAlive功能

-g filename //输出结果数据到gnuplot格式的文件中

-e filename //输出结果数据到CSV文件中

HttpWatch工具

HttpWatch工具可以用来抓取浏览器（IE）和服务器之间的通讯，并能直观的展示IE下载网页元素的顺序，花费的时间等信息

Fiddler工具

Fiddler工具是一款和HttpWatch类似的工具，完全免费

YSlow工具

YSlow（Yahoo Slow）是一个浏览器插件，通过对Web页面的分析和一些基本规则来发现Web前端的性能问题

JMeter工具

JMeter是一个提供与LR类似功能的开源性能测试工具，其特点：

- 具有很好的扩展性
- 能够很好的支持Web应用的性能测试

目录

- 什么是性能，什么是性能测试
- 性能测试工具介绍
- 目前产品的性能测试做法

目前产品性能测试的做法

- 确定目标
- 确定范围
- 确定方案
- 确定工具
- 性能测试执行
- 结果分析
- 性能测试报告

目前产品性能测试的做法

- 确定目标
  - 目标是什么？我们测试是为了什么?
  - 有没有明确的指标?
  - 在没有明确的指标的情况下，怎样才算测试到位了?

目前产品性能测试的做法

- 确定范围
  - 范围界定很重要，性能测试只能为少数的目的服务，性能测试也不能做到全覆盖。
  - 测什么，不测什么？为什么要测试?
  - 保证在有限的范围，性能是合格的就达到了目的。

目前产品性能测试的做法

- 确定方案
  - 确定性能测试的方案，多方评审。达成一致后开始测试。
  - 根据时间，重要性来排优先级，可以把不重要，不紧急的场景抛弃。
  - 方案是写个性能测试的执行者参考的，要详细和确定。多用建议设置成……，少用大概，差不多等词语。

目前产品性能测试的做法

- 确定工具
  - 在有限的环境下做性能测试，需要选择合适的工具。如性能测试用机都不能承担loadrunner，那就需要用ab来进行测试。又或者客户需要监控WAS的资源使用情况，需要使用loadrunner8.1的版本，打上fixpack4才能使用。

目前产品性能测试的做法

- 性能测试执行
  - 需要注意版本维护，脚本的独立性。
  - 各个监控数据的收集。
  - 性能测试的加压方式和性能测试脚本的合理性。
  - 严密监控服务器性能指标，注意TPS的均方差，CPU利用率和IO的利用率。包括loadrunner本身的性能情况。
  - 打开控制台看日志，保证运行的用例都是正确运行。

目前产品性能测试的做法

- 结果分析
  - 在性能测试完成某一场景后，建议分析结果集，取稳定运行的数据数据做分析。如CPU不稳定，TPS不稳定，均有可能是系统有问题，建议重新运行并根据需要进行分析。

目前产品性能测试的做法

- 性能测试报告
  - 根据要求出具性能测试报告。对内，对外，简报均可。
  - 需要评审后提交。

目录

- 什么是性能，什么是性能测试
- 性能测试工具介绍
- 目前产品的性能测试做法

第二课　问题定位与解决入门

问题定位入门

- 看数据
- 看现象
  - 看曲线
  - 看流量
- 看控制台输出
- 看线程堆栈
- 看热方法执行
- 看内存堆栈

问题定位入门

- 看数据
  - 一组用例在特定的机器上，应该能到达什么水平？大致可以估算。
  - 应用服务器CPU使用率很低！
  - TPS值很低！

在机器资源使用充分的情况下：

TPS = 强并发数 / 响应时间

问题定位入门

- 看数据
  - 数据库CPU很高

    数据库查询可能缺少索引，数据库可能配置不对

    应对方式：抓快照
    - >exec dbms_workload_repository.create_snapshot;
    - >exec dbms_workload_repository.create_snapshot;
    - @?/rdbms/admin/awrrpt.sql;

问题定位入门

- 看数据
  - 应用服务器CPU不高

    可能有线程锁，看看JVM的线程到底在做什么？

    应对方式：kill -3 <pid>

    可能有异常，大量的错误信息输出到nohup.out文件中，有问题。

问题定位入门

- 看数据
  - TPS值感觉上不合理

    检查loadrunner controller的配置

    大量的记录日志，可能导致loadruner机器IO成为瓶颈，不能发出足够并发的请求。

    检查loadrunner client的CPU状况，如果loadrunner成为瓶颈，建议添加机器进行压力测试。

    weblogic 的http日志开启，EOS日志开启等。

    可能loadrunner机器本身的线程数没有调优，请求发不出去，而响应收不回来。

问题定位入门

- 看现象
  - 看曲线

    曲线的均方差很大，抖动的厉害
  - 看流量

    网络占用非常高

问题定位入门

- 看现象
  - TPS曲线抖动的很厉害，一下跌到0，时而会冲很高

线程锁，用例脚本复查。也可能是网络瓶颈，可以用perfmon或 nmon来监控网络情况。

PRIMETON · 普元

问题定位入门

- 看现象
  - TPS曲线稳定下跌，看走势，可能时间拉长，会跌倒0

    可能有内存溢出，建议把内存调小，然后间隔时间打内存dump，通过工具分析找到瓶颈。

问题定位入门

- 看控制台输出
  - 是不是用例本身会在高并发下出错?
    报错了当然就不合理了。

PRIMETON · 普元

问题定位入门

- 看线程堆栈
- 看热方法执行

以上2个查看均可以使用jrockit来做一些事情。

问题定位入门

- 看线程堆栈

  线程堆栈可以看出来两个问题：

  1）当前大量的线程在忙什么？

  2）有没有线程等待

  建议在分析问题时，使用jrockit，jrockit的线程堆栈比较清晰。

问题定位入门

- 看热方法

  调用最多的方法，优化的价值最大

  在JVM中，调用最多的方法，用jrmc能抓去出来内存切片，可以看到方法调用的占比。对热方法进行性能优化，对提高整体性能有比较大的帮助。

问题定位入门

- 看内存堆栈
  - Jmap 获取内存堆栈做比对
  - Visualvm 获取内存堆栈
  - Jconsole 获取内存堆栈
  - 加参数获取内存堆栈

    -XX:+HeapDumpOnCtrlBreak

    -XX:HeapDumpPath=/home/liu-xj/

    -XX:-HeapDumpOnOutOfMemoryError

    -XX:HeapDumpPath=./java_pid<pid>.hprof
- 隔一段时间打印内存堆栈，看对象大小变化

使用工具总结

- 使用的工具
  - Mission control
  - Visual vm
  - Jdk 自带工具
  - MAT
  - Loadrunner
  - 数据库快照
- 分析的场景
  - 线程锁
  - 热方法
  - 内存溢出

在产品PK中常用的调优方法总结

- 操作系统
- 应用服务器
- 数据库
- 公司产品
- Loadrunner
- 用例

在产品PK中常用的调优方法总结

- 操作系统
  - 单个用户最大并发进程数
  - 单个进程最大并发线程数
  - Open file 数

在产品PK中常用的调优方法总结

- 应用服务器
  - 线程池
  - 日志
  - 连接池
  - Jdk，根据交易类型选择JDK和JVM参数

    我的经验：

    jrockit在处理网络会比sun好点。

    IBM jdk 在处理位运算比sun 好点。

    使用IBM jdk就一定要记得补丁。

在产品PK中常用的调优方法总结

- 数据库
  - SGA,PGA调整
  - Redo.log的存放路径，大小
  - 数据文件的自增长

  独立应用服务器一般建议SGA的大小为应用服务器内存总量的40%。，PGA的大小与并发的数据库连接有关，数据操作频繁但数据量不大的情况下，每个用户占用内存3-5M左右

在产品PK中常用的调优方法总结

- 公司产品
  - DAS步长，1000起，PK，比较极限的场景100000。
  - 日志级别，建议error，我们的日志非异步，大量的日志记录会让拖累TPS。
  - 监控关闭。

PRIMETON · 普元

在产品PK中常用的调优方法总结

- Loadrunner
  - 指定MDRV.EXE使用进程数
    - 在loadrunner中，默认的是每50个vuser会使用一个mdrv.exe进程，但是有些时候vuser中的使用的线程太多就会使测试程序的效率特别低，这时候就希望能够可以多起几个mdrv.exe的进程，每个进程中的vuser数量少一点，通过更改HP\LoadRunner\dat\protocols\ Java_protocols.lrp文件。在[Vugen]下面新加一条MaxThreadPerProcess=要设置的vuser数量 这样每个mdrv.exe进程中的vuser数量就是你上面设置的数量了。
  - 更改对外连接的端口最大值
  - 关闭日志等

在产品PK中常用的调优方法总结

- 用例
  - 单例
  - 缓存

Thanks!

Q&A